

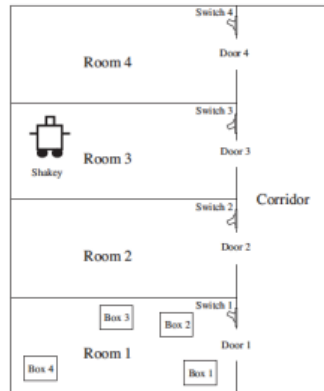
ΠΡΟΗΓΜΕΝΑ ΘΕΜΑΤΑ ΤΕΧΝΗΤΗΣ ΝΟΗΜΟΣΥΝΗΣ

Φυλλάδιο 1^ο

Άσκηση 1: Πρόβλημα του ρομπότ Shakey

1^η Άσκηση.

Το πρότυπο του STRIPS είχε προταθεί για τον έλεγχο του ρομπότ Shakey. Στην παρακάτω εικόνα φαίνεται ο κόσμος του Shakey, που αποτελείται από 4 δωμάτια που συνδέονται με ένα διάδρομο, με κάθε δωμάτιο να έχει μια πόρτα και ένα διακόπτη.



Ο Shakey στην πραγματικότητα μπορεί να μετακινείται από μια θέση στην άλλη, να σπρώχνει μετακινήσιμα αντικείμενα (π.χ. τα κουτιά) και να ανάβει / σβήνει τους διακόπτες. Αν θεωρήσουμε ότι οι ενέργειες που μπορεί να εκτελέσει ο Shakey είναι οι εξής:

1. Η ενέργεια Μετακίνηση(χ, ψ), που απαιτεί ο Shakey να βρίσκεται στη θέση χ και τα χ, ψ να είναι θέσεις στο ίδιο δωμάτιο.

Σημειώστε ότι (α) στα δωμάτια υπάρχουν διακριτές θέσεις «ενδιαφέροντος» και (β) Μια πόρτα που βρίσκεται μεταξύ δύο δωματίων ανήκει και στα δύο δωμάτια.

2. Η ενέργεια Σπρώξιμο(A, χ, ψ), όπου A ένα κουτί και χ, ψ θέσεις στο ίδιο δωμάτιο με ανοικτό τον διακόπτη. Το κατηγορημα κουτί(χ) διασφαλίζει ότι το χ είναι κουτί.
3. Το Σκαρφάλωμα(A) και η Κατάβαση(A) αφορούν το σκαρφάλωμα και την κατάβαση από κουτί. Το κατηγορημα ΠάνωΣε(χ) αφορά στο που βρίσκεται ο Shakey και υπάρχει και η σταθερά Πάτωμα
4. Η ενέργεια Άναμμα(χ) και Σβήσιμο(χ) ενός διακόπτη μπορούν να επιτευχθούν μόνο αν ο Shakey βρίσκεται στη θέση του διακόπτη.

A(10%). Μοντελοποιήστε τον κόσμο του Shakey ώστε να μπορείτε να περιγράψετε τις καταστάσεις που ενδιαφέρουν τον πράκτορά μας.

B (10%). Περιγράψτε τις ενέργειες του Shakey σε STRIPS.

Γ (50%). Κατασκευάστε ένα πλάνο ώστε ο Shakey να μεταφέρει το Box 2 στο Δωμάτιο 2 χρησιμοποιώντας (α) τον αλγόριθμο POP (β) τον αλγόριθμο GraphPlan (2 επίπεδα καταστάσεων και 2 ενεργειών θεωρώντας μόνο τα δωμάτια 1 και 3, και το κουτί 1).

A.

Για την μοντελοποίηση του κόσμου του Shakey ορίζουμε:

- **Κατηγορήματα:**

- ***at(Shakey, x)*** - Ο Shakey βρίσκεται στη θέση x
- ***at(κουτί, x)*** - Το κουτί βρίσκεται στη θέση x
- ***door(x, y)*** - Πόρτα συνδέει δωμάτια x και y
- ***switch(x, status)*** - Ο διακόπτης x είναι ανοιχτός/κλειστός
- ***in_room(x, r)*** - Η θέση x ανήκει στο δωμάτιο r
- ***on(Shakey, x)*** - Ο Shakey βρίσκεται πάνω στο αντικείμενο x (ή Πάτωμα)

B.

1. Μετακίνηση(x,y)

- Προϋποθέσεις:
 - *at(Shakey, x)*
 - *in_room(x, r)*
 - *in_room(y, r)*
 - *on(Shakey, Πάτωμα)* (προστέθηκε)
- Προσθήκες: *at(Shakey, y)*
- Διαγραφές: *at(Shakey, x)*

2. Σπρώξιμο(A, x, y)

- Προϋποθέσεις:
 - *at(Shakey, x)*
 - *at(A, x)*
 - *κουτί(A)*
 - *in_room(x, r)*
 - *in_room(y, r)*
 - *switch(s, on)* (για το δωμάτιο)
 - *on(Shakey, Πάτωμα)* (προστέθηκε)

- Προσθήκες: $at(A, y)$, $at(Shakey, y)$
- Διαγραφές: $at(A, x)$, $at(Shakey, x)$

3. **Σκαρφάλωμα(A)** (προστέθηκε)

- Προϋποθέσεις:
 - $at(Shakey, x)$
 - $at(A, x)$
 - $κουτί(A)$
 - $on(Shakey, Πάτωμα)$
- Προσθήκες: $on(Shakey, A)$
- Διαγραφές: $on(Shakey, Πάτωμα)$, $at(Shakey, x)$

4. **Κατάβαση(A)** (προστέθηκε)

- Προϋποθέσεις:
 - $on(Shakey, A)$
 - $at(A, x)$
- Προσθήκες: $on(Shakey, Πάτωμα)$, $at(Shakey, x)$
- Διαγραφές: $on(Shakey, A)$

5. **Άναμμα(x)** (προστέθηκε)

- Προϋποθέσεις:
 - $at(Shakey, x)$
 - $switch(x, off)$
- Προσθήκες: $switch(x, on)$
- Διαγραφές: $switch(x, off)$

6. **Σβήσιμο(x)** (προστέθηκε)

- Προϋποθέσεις:
 - $at(Shakey, x)$

- switch(x, on)
- Προσθήκες: switch(x, off)
- Διαγραφές: switch(x, on)

Σημαντικές σημειώσεις:

- Προστέθηκε το κατηγορήμα on(Shakey, Πάτωμα) σε σχετικές ενέργειες
- Οι ενέργειες σκαρφαλώματος/κατάβασης αφορούν αποκλειστικά κουτιά (κουτί(A))
- Οι ενέργειες διακοπείας απαιτούν τον Shakey να βρίσκεται ακριβώς στη θέση του διακόπτη
- Η μετακίνηση και το σπρώξιμο απαιτούν οι θέσεις να είναι στο ίδιο δωμάτιο

Γ.

(α) Με αλγόριθμο POP

1. **Αρχική κατάσταση**: at(Shakey,P1), at(Box2,P3), in_room(P1,Room1), in_room(P3,Room3)
2. **Στόχος**: at(Box2,P2), in_room(P2,Room2)
3. **Βήματα**:
 - Μετακίνηση(P1,D1)
 - Μετακίνηση(D1,P3)
 - Άναμμα(S1)
 - Σπρώξιμο(Box2,P3,D3)
 - Μετακίνηση(D3,P2)

(β) Με GraphPlan

Περιορισμοί Προβλήματος:

- Θεωρούμε μόνο τα δωμάτια 1 και 3

- Μόνο το κουτί 1 (Box1)
- 2 επίπεδα καταστάσεων και 2 επίπεδα ενεργειών

GraphPlan Expansion:

Level 0 (Initial State):

- Καταστάσεις:
 - at(Shakey, P1)
 - at(Box1, P1)
 - in_room(P1, Room1)
 - in_room(D1, Room1), in_room(D1, Room3) (πόρτα)
 - switch(S1, off)
 - on(Shakey, Πάτωμα)

Level 1 (Actions):

- Δυνατές Ενέργειες:
 1. Move(P1, D1)
 - Προϋποθέσεις: at(Shakey,P1), in_room(P1,Room1), in_room(D1,Room1)
 - Εφέ: at(Shakey,D1), $\neg$ at(Shakey,P1)
 2. TurnOn(S1)
 - Προϋποθέσεις: at(Shakey,P1), switch(S1,off)
 - Εφέ: switch(S1,on), $\neg$ switch(S1,off)
- MOTX (mutex) Σχέσεις:
 - Move(P1,D1) και TurnOn(S1) είναι mutually exclusive (απαιτούν και οι δύο τον Shakey στο P1)

Level 1 (State):

- Νέες Καταστάσεις:
 - at(Shakey, D1) (από Move)
 - switch(S1, on) (από TurnOn)

- ο Παλιές καταστάσεις που διατηρούνται:

- at(Box1, P1)
- in_room(P1, Room1)
- on(Shakey, Πάτωμα)

Level 2 (Actions):

- Δυνατές Ενέργειες:

1. Move(D1, P3) (P3 στο Room3)

- Προϋποθέσεις: at(Shakey,D1), in_room(D1,Room3), in_room(P3,Room3)

2. Push(Box1, P1, D1)

- Προϋποθέσεις:
 - at(Shakey,P1) (ΔΕΝ ισχύει τώρα - Shakey είναι στο D1)
 - at(Box1,P1)
 - switch(S1,on)
 - in_room(P1,Room1), in_room(D1,Room1)
- => Αδύνατη ενέργεια σε αυτό το επίπεδο

3. TurnOff(S1)

- Προϋποθέσεις: at(Shakey,P1), switch(S1,on) (Shakey δεν είναι στο P1)

- MOTX Σχέσεις:

- ο Καμία, αφού μόνο η Move(D1,P3) είναι εφικτή

Level 2 (State):

- Εφικτή μόνο η ενέργεια Move(D1, P3)
- Τελική Κατάσταση:
 - ο at(Shakey, P3)
 - ο at(Box1, P1) (αμετάβλητο)
 - ο switch(S1, on)

- in_room(P3, Room3)

2η Άσκηση.

Μετατρέψτε τον αλγόριθμο POP ώστε να χειρίζεται πόρους (αναλώσιμους ή μη). Δείξτε το στα παράδειγμα για τη συναρμολόγηση αυτοκινήτων στις διαφάνειές του μαθήματος θεωρώντας

A (15%) μη-αναλώσιμους πόρους (βαρούλκα, επιθεωρητές, σταθμοί τροχών)

B (15%) αναλώσιμους πόρους (βίδες: συγκεκριμένα υποθέστε 'ότι υπάρχουν 100 βίδες, και η μια μηχανή E_1 απαιτεί 40 και η άλλη μηχανή E_2 απαιτεί 60 βίδες. Ορίστε τις ενέργειες με βάση τις απαιτήσεις και τις επιδράσεις τους στον αναλώσιμο πόρο «βίδες» (στις επιδράσεις μπορείτε να χρησιμοποιούνται τα σύμβολα +, -)

Γ (10%) Με βάση τον τροποποιημένο σας αλγόριθμο δημιουργήστε ένα πλάνο για την συναρμολόγηση των αυτοκινήτων, δίχως χρονοπρογραμματισμό ενεργειών

Δ. (10%)

- Αν οι αρχική ποσότητα των διαθέσιμων βιδών είναι 90, ο αλγόριθμός σας αναγνωρίζει το αδιέξοδο; Εξηγήστε με λεπτομέρεια πως γίνεται αυτό.
- Τι θα προτείνατε για την επίλυση του προβλήματος αυτού;

Ερώτημα A: Μη-αναλώσιμοι πόροι

1.1. Ορισμός πόρων

- **Βαρούλκα:** 1 μονάδα
- **Σταθμοί τροχών:** 1 μονάδα
- **Επιθεωρητές:** 2 μονάδες

1.2. Διαχείριση στο POP

- Κάθε action δηλώνει πόσες μονάδες του κάθε πόρου δεσμεύει κατά την εκτέλεσή του.
- Στο στάδιο της επίλυσης conflicts (συγκρούσεων) μεταξύ causal links, αν δύο actions απαιτούν τον ίδιο πόρο, προστίθεται ordering constraint ώστε να μη συμπίπτουν χρονικά (π.χ. $S_1 < S_4$ ή $S_4 < S_1$) και διασφαλίζεται ότι δεν υπερβαίνεται η διαθέσιμη ποσότητα.

Ερώτημα B: Αναλώσιμοι πόροι – Βίδες

2.1. Αρχική ποσότητα: 100 βίδες

2.2. Actions & Κατανάλωση

Action	Κατανάλωση βιδών	Σημείωση
ΠροσθήκηΜηχανής(E1, c)	– 40	E1 χρειάζεται 40 βίδες
ΠροσθήκηΜηχανής(E2, c)	– 60	E2 χρειάζεται 60 βίδες

2.3. Open-condition για πόρους

- Για κάθε action που καταναλώνει q βίδες, εισάγουμε open condition:
Βίδες $\geq q$

2.4. Επιδράσεις (Effects) πόρου

- Κάθε φορά που επιλέγεται action, μειώνουμε το μερικό balance κατά q .
- Αν $\text{balance} - q < 0$, το μονοπάτι απορρίπτεται ως μη-συνεπές (deadlock).

Ερώτημα Γ: Πλάνο χωρίς χρονοπρογραμματισμό

3.1. STEPS (Partial-Order Plan)

- S₁**: ΠροσθήκηΜηχανής(E1, C1)
- S₂**: ΠροσθήκηΤροχών(W1, C1)
- S₃**: Επιθεώρηση(C1)
- S₄**: ΠροσθήκηΜηχανής(E2, C2)
- S₅**: ΠροσθήκηΤροχών(W2, C2)
- S₆**: Επιθεώρηση(C2)

3.2. Causal links

- $S_1 \rightarrow \text{ΜηχανήΕντός}(C1) \rightarrow S_2$
- $S_2 \rightarrow \text{ΤροχοίΕπί}(C1) \rightarrow S_3$
- $S_4 \rightarrow \text{ΜηχανήΕντός}(C2) \rightarrow S_5$
- $S_5 \rightarrow \text{ΤροχοίΕπί}(C2) \rightarrow S_6$

3.3. Ordering constraints

- $\text{Start} < S_1 < S_2 < S_3 < \text{Finish}$
- $\text{Start} < S_4 < S_5 < S_6 < \text{Finish}$
- **Βαρούλκα(1):** S_1 και S_4 αποκλείονται ταυτόχρονα $\Rightarrow S_1 < S_4$.
- **Σταθμοί τροχών(1):** S_2 και S_5 αποκλείονται $\Rightarrow S_2 < S_5$.
- **Επιθεωρητές(2):** S_3, S_6 μπορούν παράλληλα.

Ερώτημα Δ: Deadlock με 90 βίδες & Επίλυση

4.1. Διάγνωση αδιεξόδου

- Αν θέσουμε αρχικό Βίδες = 90, τότε:
 - Με S_1 (-40) και S_4 (-60) $\rightarrow$ νέα balance = -10 (< 0).
 - Κατά τον έλεγχο open condition “Βίδες $\geq q$ ” για S_4 ($q=60$), αποτυγχάνει και το πλάνο απορρίπτεται.

4.2. Προτάσεις επίλυσης

1. Νέα action παραγγελίας βιδών

ΠαραγγελίαΒιδών(n):

Precond: –

Effects: Βίδες += n

2. Εναλλακτικά εξαρτήματα

- Χρήση clips αντί βιδών

3. Δυναμικός επαναπρογραμματισμός

- Μετά την ανίχνευση shortage, εισαγωγή ΠαραγγελίαΒιδών και επανεκτέλεση του planner